

Header Files & Setup

```
#include <iostream>
#include <omp.h>
#include <limits.h>
#include <algorithm>
```

```
using namespace std;
```

- `#include <iostream>`: Includes the standard input-output stream library, allowing us to use `cout` to print text to the console.
- `#include <omp.h>`: Includes the **OpenMP** library. This is the engine that allows us to run code in parallel (on multiple CPU cores).
- `#include <limits.h>`: Provides constants for integer limits. We use `INT_MIN` (the smallest possible integer) as a starting point when looking for the maximum value.
- `#include <algorithm>`: (Optional here) Usually includes standard algorithms like `std::max`, though we manually implement the logic in this script.
- `using namespace std;`: Tells the compiler to use the "standard" namespace so we can type `cout` instead of `std::cout`.

Memory Management Functions

```
// Allocate n x n matrix
int** allocateMatrix(int n) {
    int** mat = new int* [n];
    for (int i = 0; i < n; i++) {
        mat[i] = new int[n];
    }
    return mat;
}
```

- `int** mat = new int* [n];`: Creates an array of *pointers* (rows). `mat` is a "pointer to a pointer."
- `for (int i = 0; i < n; i++)`: Loops through each row index.
- `mat[i] = new int[n];`: For each row pointer, we allocate an actual array of `n` integers. This creates the 2D grid in memory.
- `return mat;`: Returns the address of this grid so we can use it.

```
// Free matrix memory
void freeMatrix(int** mat, int n) {
    for (int i = 0; i < n; i++) {
        delete[] mat[i];
    }
    delete[] mat;
}
```

- `delete[] mat[i];`: Deletes the actual array of integers for each row (cleaning up the "columns").
- `delete[] mat;`: After all rows are deleted, this deletes the array of row pointers.

- Why? If we don't do this, the computer's RAM will fill up (Memory Leak) because we run this many times.

1. Matrix Initialization

```
// Initialize matrix
void fillMatrix(int** mat, int n, int val) {
    // schedule(static) is better for memory initialization
    #pragma omp parallel for collapse(2) schedule(static)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            mat[i][j] = val;
        }
    }
}
```

- `#pragma omp parallel for`: Tells the compiler "Run the following loop on multiple threads."
- `collapse(2)`: Merges the `i` (rows) and `j` (columns) loops into one giant loop of size $N \times N$. This gives threads a larger pool of work to divide.
- `schedule(static)`: Divides the work into fixed, equal chunks at the start. Since filling a matrix takes the exact same time for every cell, this is the fastest method (lowest overhead).
- `mat[i][j] = val;`: Sets the value of the cell.

2. Parallel Matrix Sum

```
long long matrixSum(int** mat, int n) {
    long long total_sum = 0;
    #pragma omp parallel for collapse(2) reduction(+:total_sum)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            total_sum += mat[i][j];
        }
    }
    return total_sum;
}
```

- `long long total_sum = 0;`: We use `long long` because adding millions of integers might exceed the limit of a standard `int`.
- `reduction(+:total_sum)`: This is the magic part.
 - It gives every thread its own **private** copy of `total_sum` initialized to 0.
 - Threads add their numbers to their private copies (preventing race conditions).
 - At the end, OpenMP adds all the private copies together into the final `total_sum`.

3. Parallel Matrix Maximum

```
int matrixMax(int** mat, int n) {
    int global_max = INT_MIN;
    #pragma omp parallel for collapse(2) reduction(max:global_max)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
```

```

        if (mat[i][j] > global_max) {
            global_max = mat[i][j];
        }
    }
}
return global_max;
}

```

- `int global_max = INT_MIN;`: Start with the smallest possible number so any value in the matrix will be larger.
- `reduction(max:global_max)`: Similar to `sum`, but for maximums. Each thread finds the max in its chunk, and OpenMP compares all the thread-local maximums to find the single largest number.

4. Optimized Matrix Multiplication (The Big One)

```

void matrixMult(int** A, int** B, int** C, int n) {
    // 1. Create a temporary matrix to store the Transpose of B
    int** B_T = allocateMatrix(n);

    // 2. Transpose Matrix B (Swap Rows and Cols)
    #pragma omp parallel for collapse(2) schedule(static)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            B_T[j][i] = B[i][j];
        }
    }
}

```

- **Why Transpose?** We flip Matrix B so that its columns become rows. This allows us to read it sequentially (fast) instead of jumping through memory (slow).
- `B_T[j][i] = B[i][j];`: The actual flip. Row `i` Col `j` becomes Row `j` Col `i`.

```

// 3. Multiply using the Transposed Matrix
#pragma omp parallel for collapse(2) schedule(static)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        int sum = 0;
        // The inner loop 'k' now glides smoothly through memory for
both matrices
        for (int k = 0; k < n; k++) {
            sum += A[i][k] * B_T[j][k];
        }
        C[i][j] = sum;
    }
}
freeMatrix(B_T, n);
}

```

- `schedule(static)`: Crucial fix. Dynamic scheduling caused overhead that made 8 threads slower. Static is perfect here because the work is even.
- `A[i][k] * B_T[j][k]`:

- $A[i][k]$: We iterate k (0, 1, 2...). These are neighbors in memory. **Fast.**
- $B_T[j][k]$: We iterate k . Because we transposed B , these are *also* neighbors in memory. **Fast.**
- Before optimization, this was $B[k][j]$, which required huge jumps in memory.

5. The Main Loop (Test Harness)

```
int main() {
    int sizes[] = { 1500, 3000 };
    int threads[] = { 1, 2, 4, 8 };

    for (int s = 0; s < 2; s++) { // Loop over sizes (1500, then 3000)
        int n = sizes[s];
        cout << "Processing Matrix Size: " << n << " x " << n << endl;

        // Allocate and Fill Matrices once per size
        int** A = allocateMatrix(n);
        int** B = allocateMatrix(n);
        int** C = allocateMatrix(n);
        fillMatrix(A, n, 1);
        fillMatrix(B, n, 2);

        for (int t = 0; t < 4; t++) { // Loop over thread counts (1, 2, 4,
8)
            int num_threads = threads[t];
            omp_set_num_threads(num_threads); // Set active threads for
next region

            cout << "    Number of Threads = " << num_threads << endl;
            double start, end;

            // --- Benchmark Sum ---
            start = omp_get_wtime(); // Start Clock
            long long sum = matrixSum(A, n);
            end = omp_get_wtime(); // Stop Clock
            cout << "    Sum Result = " ... ;

            // --- Benchmark Max ---
            // (Same timer logic)

            // --- Benchmark Mult ---
            // (Same timer logic)
        }

        // Clean up memory before starting the next matrix size
        freeMatrix(A, n);
        freeMatrix(B, n);
        freeMatrix(C, n);
    }
    return 0;
}
```

- The `main` function is strictly sequential (it doesn't have `#pragma omp parallel`). It controls the experiment, running one test configuration after another.
- `omp_set_num_threads(num_threads)`: This tells the OpenMP library "For the next parallel loop you see, launch exactly this many threads." This allows us to compare 1 vs 8 threads fairly.